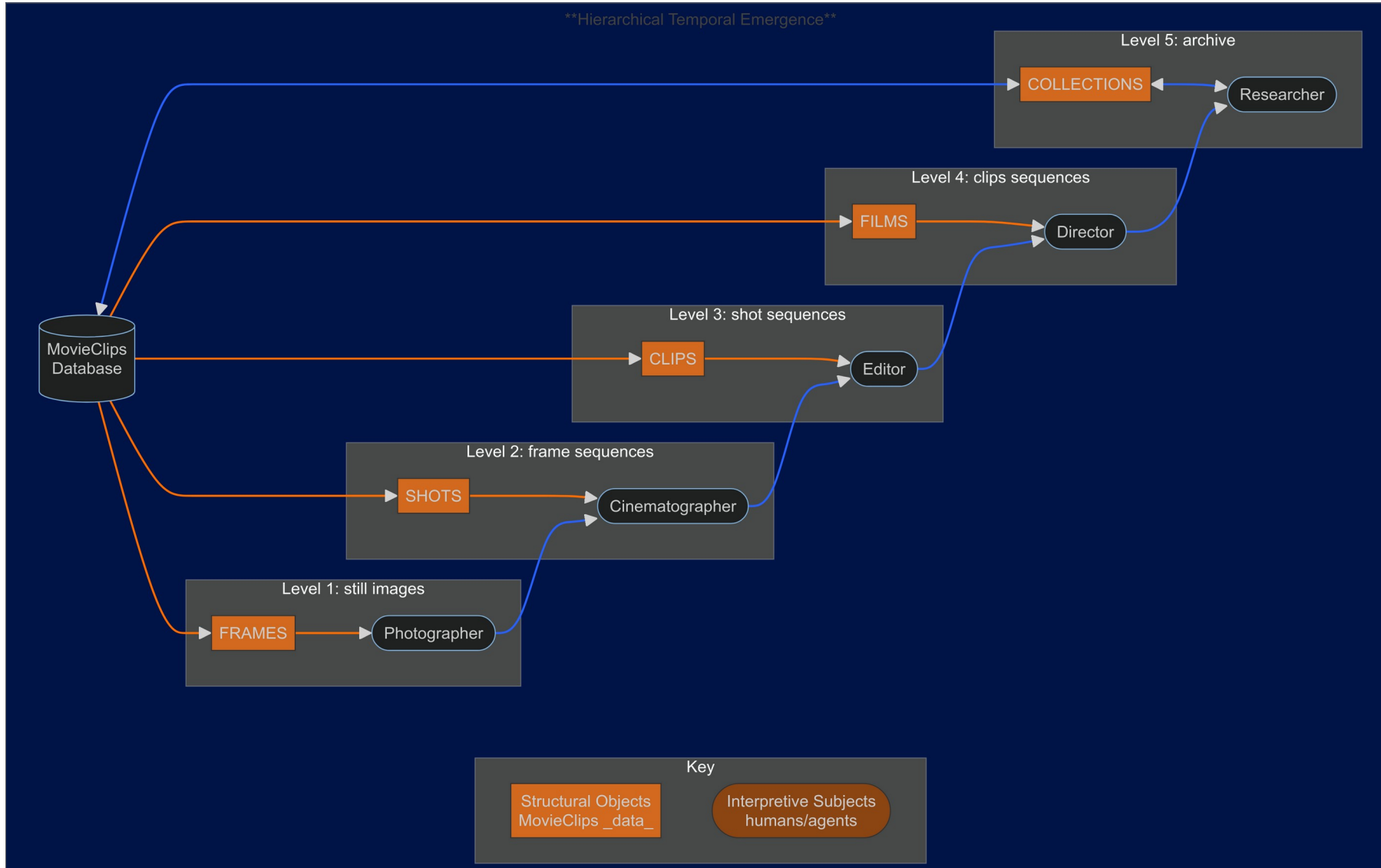


FrameSense – Tech Overview

Geoffroy Noël, May 2026



1. Collection

2. Video

(e.g. Film, Programme)

3. Clip

(e.g. Sequence, Scene)

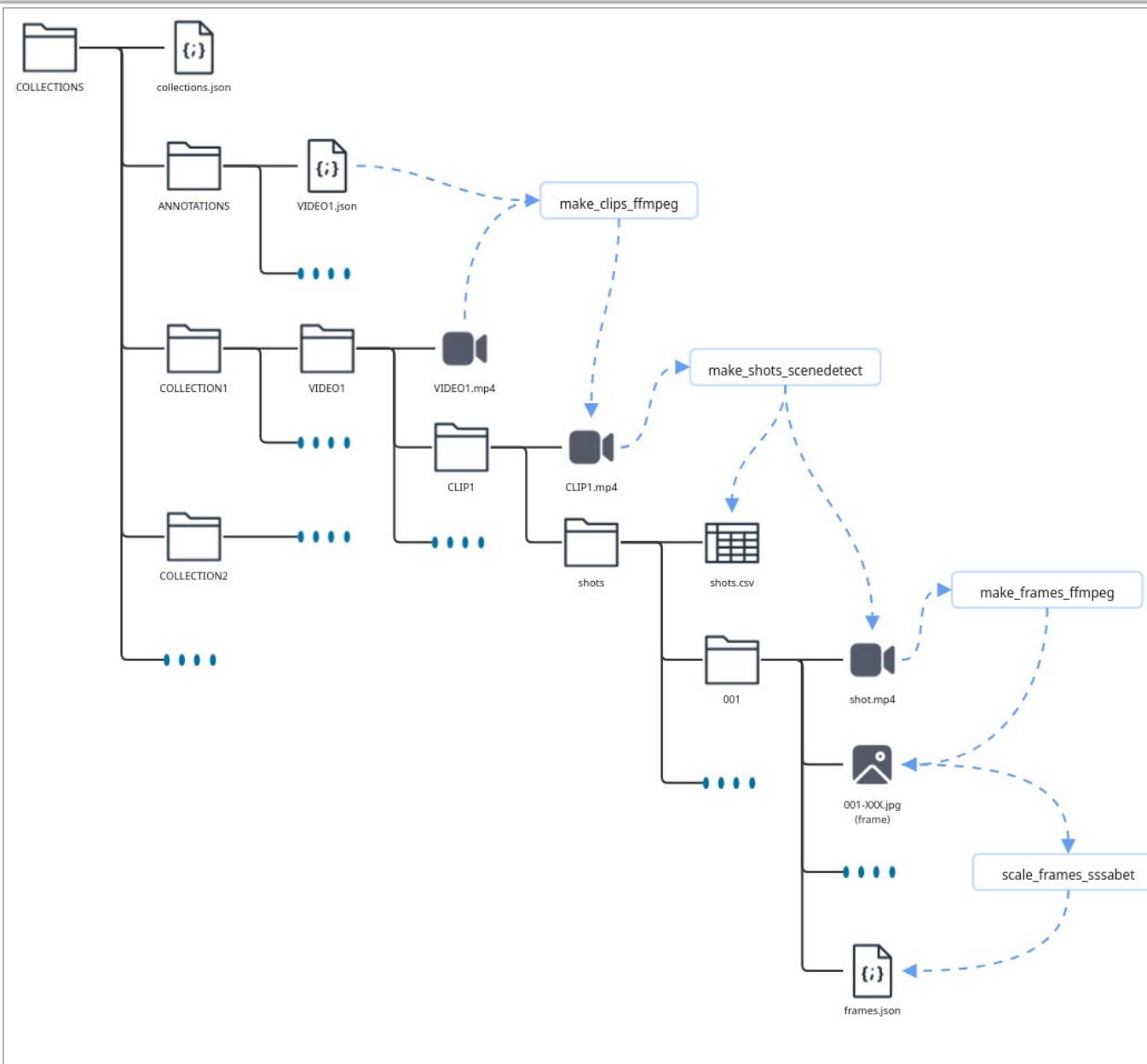
4. Shot

(and transitions, e.g. cuts)

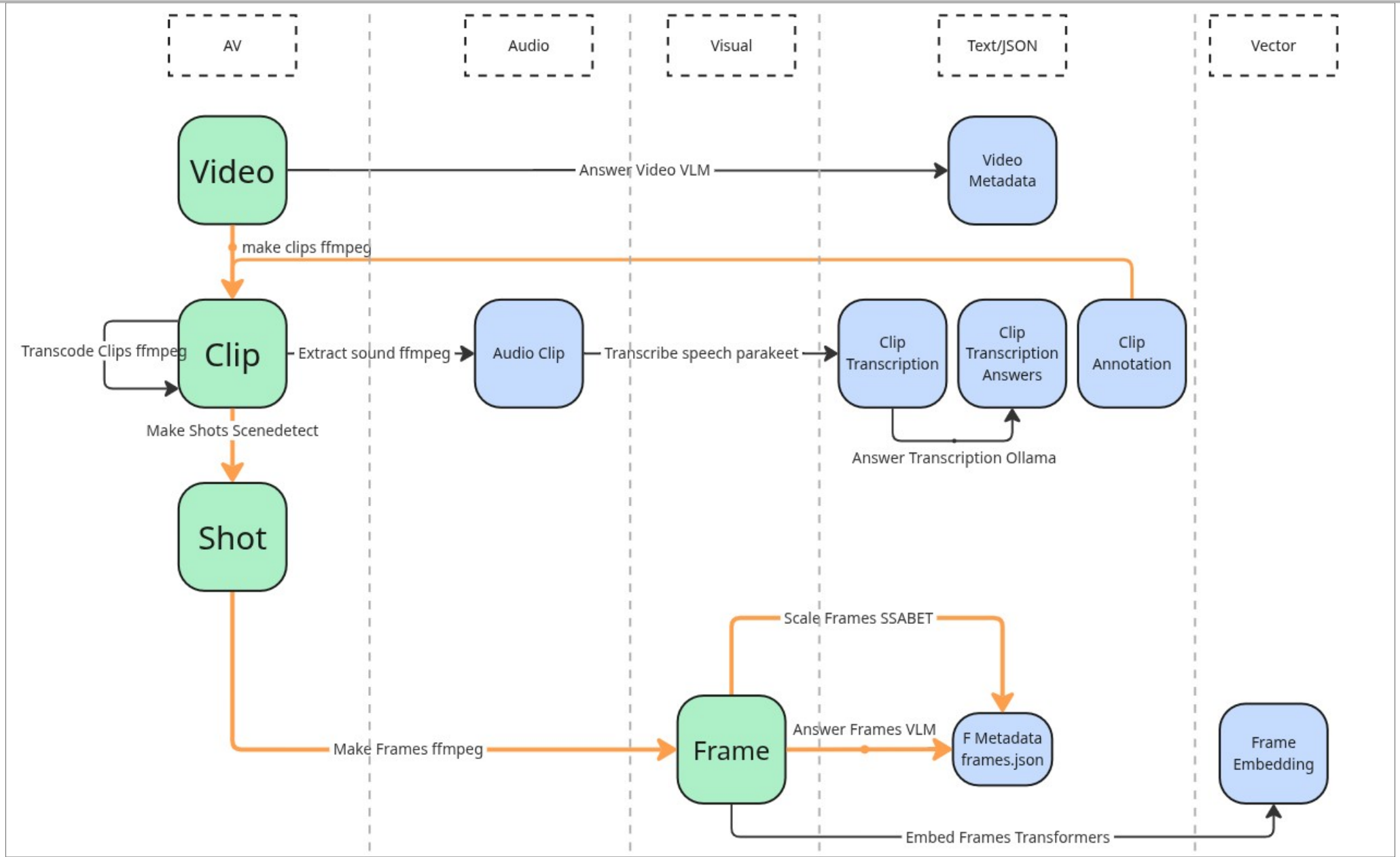
5. Frame

(Still)

Folder structure



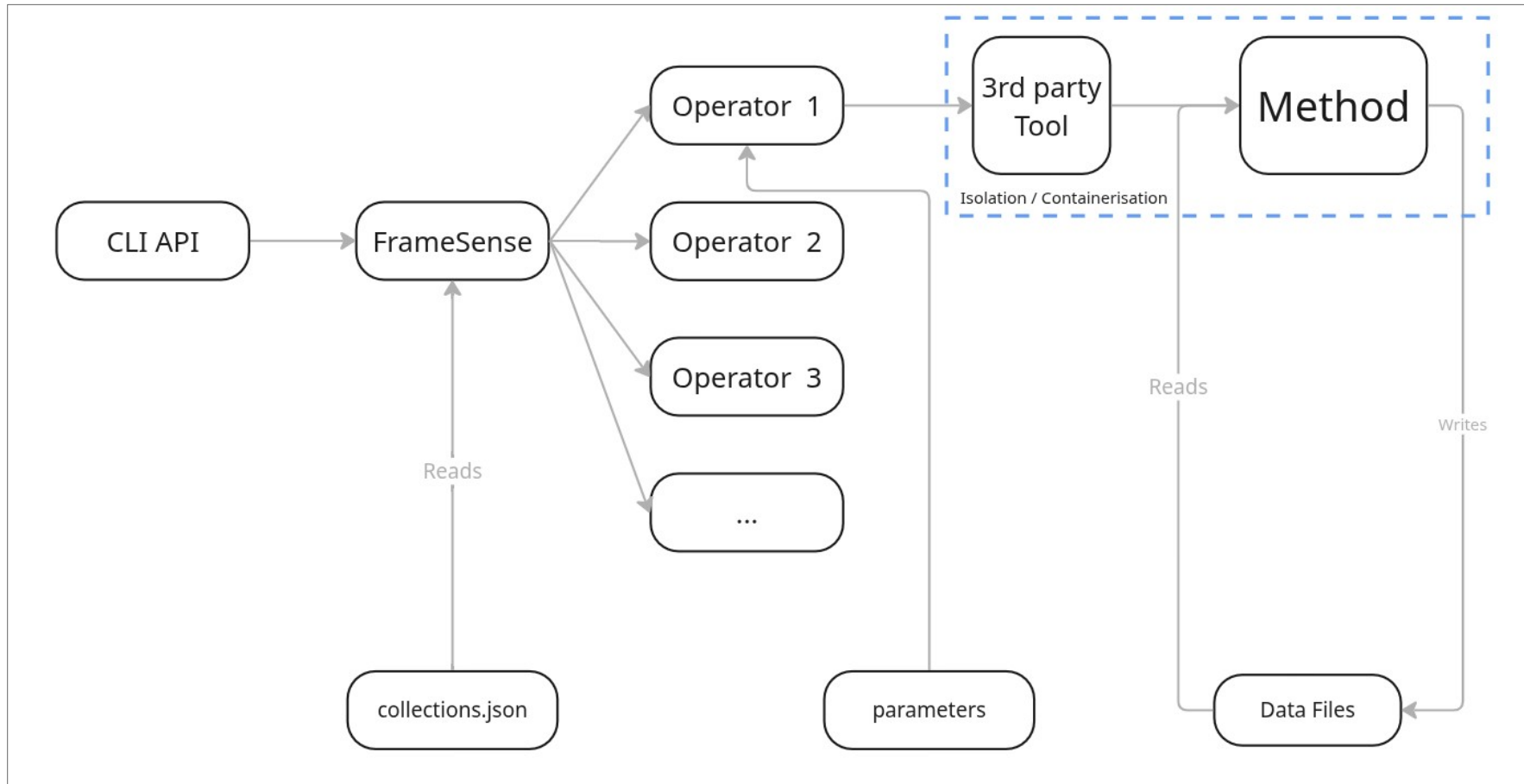
Built-in operators



collections.json

```
workshop > collections > {} collections.json > ...
```

```
1  {
2    "meta": {
3      "type": "collection",
4      "comment": "test collections",
5      "params": {
6        "answer_frames_vlm": {
7          "api_base": "http://localhost:11434/v1",
8          "model": "gemma3:4b"
9        }
10     }
11  },
12  "data": [
13    {
14      "type": "collection",
15      "id": "workshop-test",
16      "attributes": {
17        "name": "Workshop Test Collection",
18        "collector": "",
19        "description": "Test collection for DH workshop 5th May 2026",
20        "origin": "",
21        "path": "test_collection",
22        "annotations_path": "test_annotations"
23      }
24    }
25  ]
26 }
```



Python CL tool offering a range of **independent operators**. Each operator achieves one **single task** on **one slice** of the collection, and writes output back to filesystem.

Operators don't know each other, only data files.

Operators are just wrappers around existing 3rd party tools.

- 1. Modular:** open architecture so each operator has its own module and containerised dependencies; easy to extend or swap operations;
- 2. Incremental:** an operation builds on top of outputs from other operations, enabling caching, reuse of intermediate results and custom pipelines;
- 3. HPC-friendly:** non-interactive command line tool which is easy to install and run on SLURM;
- 4. Portable:** should be easy to run on different machines, including lower-end personal computers

```
operators > answer_frames_vlm > {} params.json > ...
```

```
1  {
2    "api_base": "http://localhost:11434/v1",
3    "api_key": "",
4    "model": "gemma3:4b",
5    "seed": 1,
6    "frame_filter": "middle",
7    "context_length": 2048,
8    "questions": {
9      "describe_short": {
10       "question": "describe this image in one short sentence"
11     }
12   },
13   "prompt_template_MULTILINES": [
14     "{question}"
15   ],
16   "think": false
17 }
```

A parameter value comes from:

1. environment variable
(ANSWER_FRAMES_VLM_MODEL="qwen3.5:4b")
2. variable in your .env file
3. value under
meta.params.answer_frames_vlm.model in your collection.json file
4. the default value set in the operator's
params.json file (but not user-editable)

What could be improved?

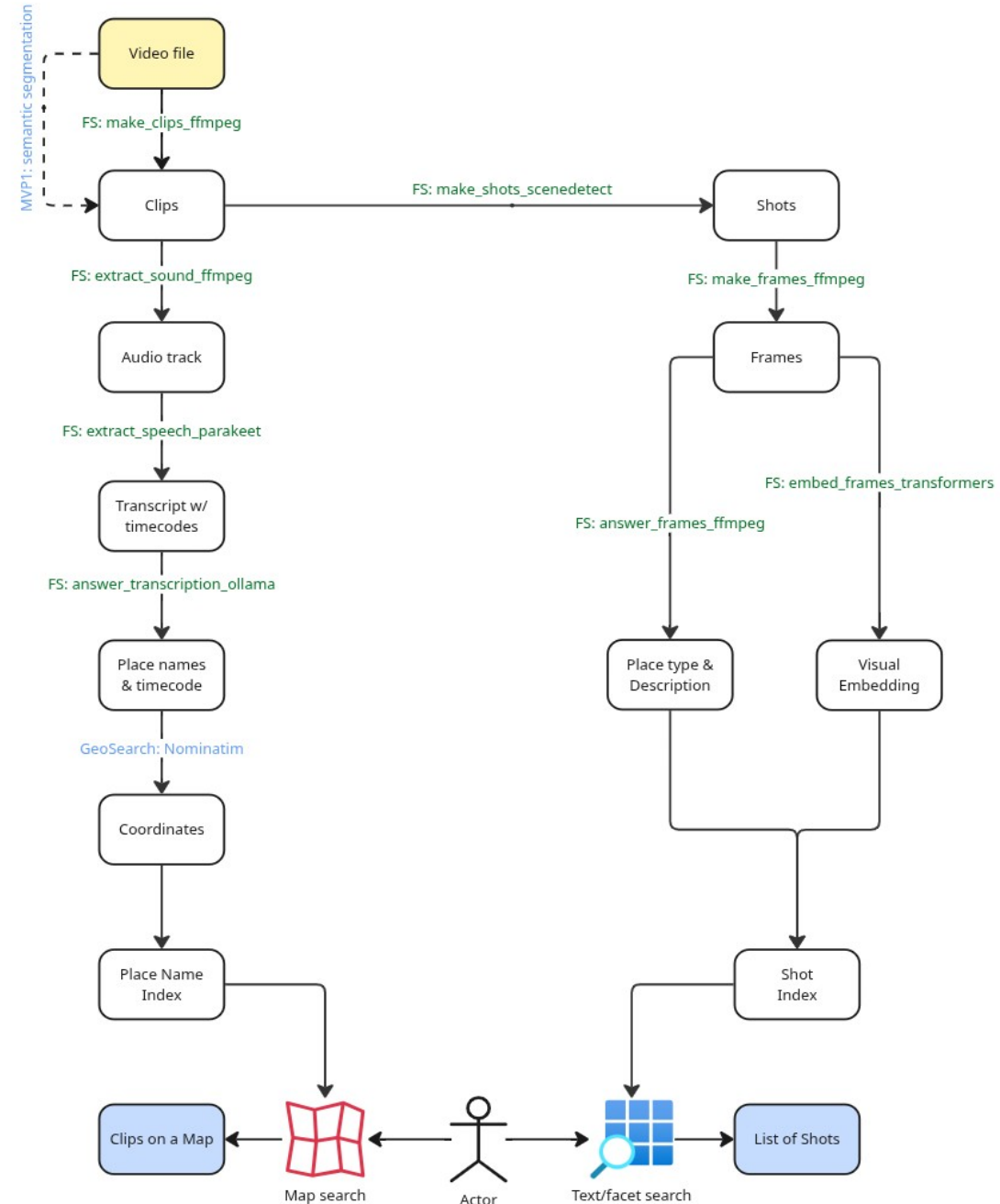
1. Openness to **third party operators** [gh-14]
2. **Performance** optimisation (e.g. enable parallelism; downscaling) [gh-18]
3. Flexibility of the collection **file structure** [gh-27]
4. Support for existing **standards** (e.g. AD, Transcription formats)
5. Operator **naming scheme**
6. **Input variant** selection
7. Organisation of **shared methods**
8. Provide a Web API or MCP toolset
9. **Clean up** unused files
10. **YOUR IDEAS** => <https://github.com/kingsdigitallab/framesense/issues>



Split the place extraction problem in two strands:

- 1) Place names from audio, left
- 2) Place types from visual, right

Each with its own (re)search interfaces.



FrameSense – Writing your own operator

Geoffroy Noël, May 2026

1. Overlay timestamps on a video or clip. The purpose is to help Video or Vision Language models more accurately locate frames or sections of interest while prompting them with a video. This could be achieved with ffmpeg.



2. Detect and extract cinematographic properties of the shots using a specialised model like [ShotVL](#).



3. Implement an alternative method for shot detection as we know PySceneDetect can be poor for some types of videos or require manual tuning for particular videos. This would complement the existing operator `make_shots_scenedetect`.

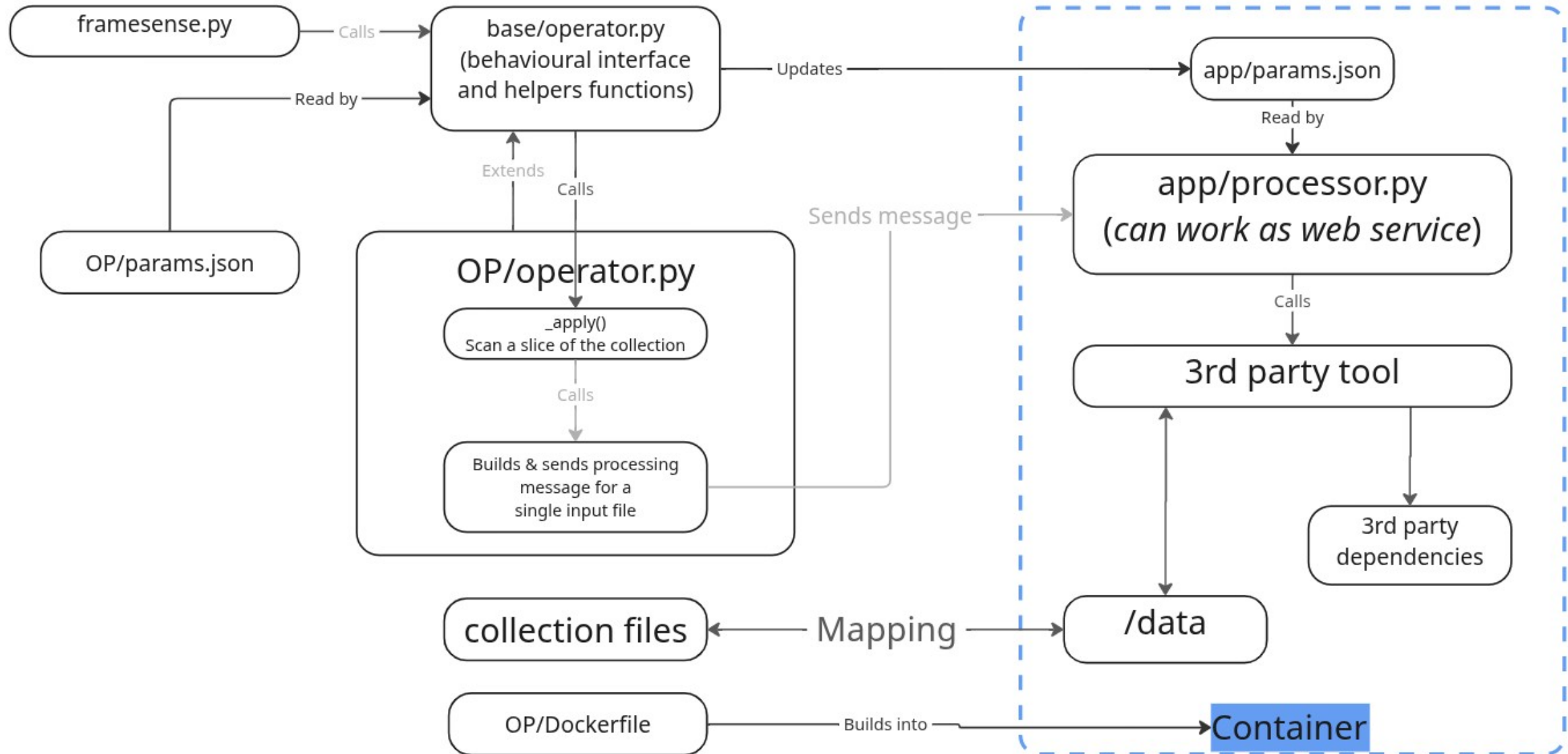
4. Develop an alternative to `answer_videos_vlm` that uses a remote Openai API instead of local transformers-based inference to answer any question about a video

5. Overlay a transcription onto the video. This can be useful to help video understanding models which can't take audio as input (e.g. Qwen), to read the subtitles from the image and greatly improve their understanding of the meaning of the unfolding events.

It is expected that each operator:

1. is functionally **minimal** ("does one thing and does it well");
2. is **atomic** (only valid and complete output are persisted);
3. implements a **single method** or strategy;
4. should **only process** the input if its output doesn't already exist;
5. uses containers to **isolate** its software dependencies;
6. works on all files at one **specific level** in the hierarchy (e.g. `make_shots` splits all your clips into shots);
7. has a **name** which reflects what it does, on what unit, with which method (e.g. `make_clips_ffmpeg`);
8. is written as a Python class within a module `operator.py` under a package which name matches the name of the operator (e.g. `operators/make_clips_ffmpeg/operator.py`);
9. **inherits** from the [base operator](#);

Anatomy of an operator



General procedure

1. Find a third party **tool** and a **method** to achieve the task
2. Determine operator **name** from method and input/output type
3. Create operator.py
4. Borrow `_apply()` from another operator working on the same input type
5. Write the unit processing function: filtering, skipping/redoing, calling
6. Containerise dependencies
7. Write processor as a command or a service
8. Test & debug on one small input
9. Parametrise with params.json
10. Document in README.md

1 & 2: timecode_clip_ffmpeg



on linux is there a command line tool to overlay time code on ev



Ask

All

Images

News

Videos

Goggles



Yes, **FFmpeg** is the standard command-line tool for this on Linux, using the **drawtext** video filter to burn a dynamic timecode into every frame during transcoding. [🔗](#)

To overlay a timecode starting from zero, you use the **timecode** parameter within the **drawtext** filter chain. You must specify the **timecode_rate** (framerate) to ensure the counter increments correctly per frame. [🔗](#)

```
ffmpeg -i input.mp4 -vf  
"drawtext=fontfile=/path/to/font.ttf:fontsize=24:fontcolor=white:box=1  
:boxcolor=black:timecode='00:00:00:00':rate=30:x=(w-tw)/2:y=10" -c:a  
copy output.mp4
```

Key parameters for customization include:

- **timecode='00:00:00:00'**: Sets the starting timecode (Hours:Minutes:Seconds:Frames). [🔗](#)
- **rate=30**: Sets the frame rate for the counter (match your video's FPS). [🔗](#)
- **x** and **y**: Controls the horizontal and vertical position of the overlay. [🔗](#)
- **fontsize**, **fontcolor**, **fontfile**: Styles the text appearance.

If you need to overlay the **original embedded timecode** from the source video rather than a new counter, you must first extract the start timecode using **ffprobe** and then pass it into the **drawtext** command, as FFmpeg does not automatically read and burn existing embedded timecode streams in a single pass. [🔗](#)

AI-generated answer. Please verify critical facts.

Task description

Tool = ffmpeg

Tool invocation

Potential
parameters

3 & 4. operator.py & _apply()

We borrow & adapt the code from another operator that works from the same units (clips):
make_shots_scenedetects.

```
8
9 class MakeShotsSceneDetect(Operator):
10     '''Extract shots from clips using PySceneDetect'''
11
12     def get_supported_arguments(self):
13         ret = super().get_supported_arguments()
14         ret['redo'] = True
15         ret['filter'] = True
16         return ret
17
18     def _apply(self):
19         ret = None
20
21         for col in self.context['collections']:
22             for video_folder_path in col['attributes']['path'].iterdir():
23                 if video_folder_path.is_dir():
24                     for clip_folder_path in video_folder_path.iterdir():
25                         if clip_folder_path.is_dir():
26                             clip_path = self._get_video_file_path(clip_folder_path, True)
27                             if clip_path:
28                                 self._make_shots(clip_path)
29
30         return ret
31
32     def _make_shots(self, clip_path: Path):
33         if not self._is_path_selected(clip_path):
34             return
```

TimecodeClipFFMPEG

Only set to True later
if your operator
actually support redo
& filter

Same scanning
logic

Rename that method,
e.g. _timecode_clip()

5. _timecode_clip()

```
34
35 def _timecode_clip(self, clip_path: Path):
36     if not self._is_path_selected(clip_path):
37         return
38
39     output_path = clip_path.parent / f'{clip_path.stem}_timecoded{clip_path.suffix}'
40
41     if not self._is_redo() and output_path.exists():
42         return
43
44     self._log(output_path)
45
46     fontsize = self.get_param('fontsize')
47     rate = self._get_video_frame_rate(clip_path)
48     vf = (
49         f"drawtext="
50         f"text='':"
51         f"timecode='00\\:00\\:00\\:00':"
52         f"rate={rate}:"
53         f"font='DejaVu Sans':"
54         f"x=10:y=10:"
55         f"fontsize={fontsize}:"
56         f"fontcolor=white:"
57         f"box=1:"
58         f"boxcolor=black@0.5:"
59         f"boxborderw=5"
60     )
61     command = [
62         'ffmpeg', '-y', '-i', clip_path,
63         '-vf', vf,
64         '-c:a', 'copy',
65         output_path,
66     ]
67     self._run_in_operator_container(command, [clip_path.parent, '/data'], same_user=True)
68
```

Input filtering logic (-f)

Skipping/redo (-r) logic

Get value of operator
parameters

Gather information from the
input file
(frame rate with ffprobe)

Construct message with
arguments for the tool
(ffmpeg)

Pass the message
through the container

ffmpeg is our primary dependency. We need to isolate it inside a container.

We can borrow a Dockerfile from another ffmpeg operator (e.g. `make_clips_ffmpeg`).

And adapt it to install one **font** (dejavu) we'll use for the timecode overlay.

```
operators > timecode_clip_ffmpeg > Dockerfile > ...
```

```
1 FROM linuxserver/ffmpeg
2 RUN apt-get update && apt-get install -y --no-install-recommends fonts-dejavu-core && rm
   -rf /var/lib/apt/lists/*
3 ENTRYPOINT []
4
```

In this case ffmpeg gives us all we need for the processing.

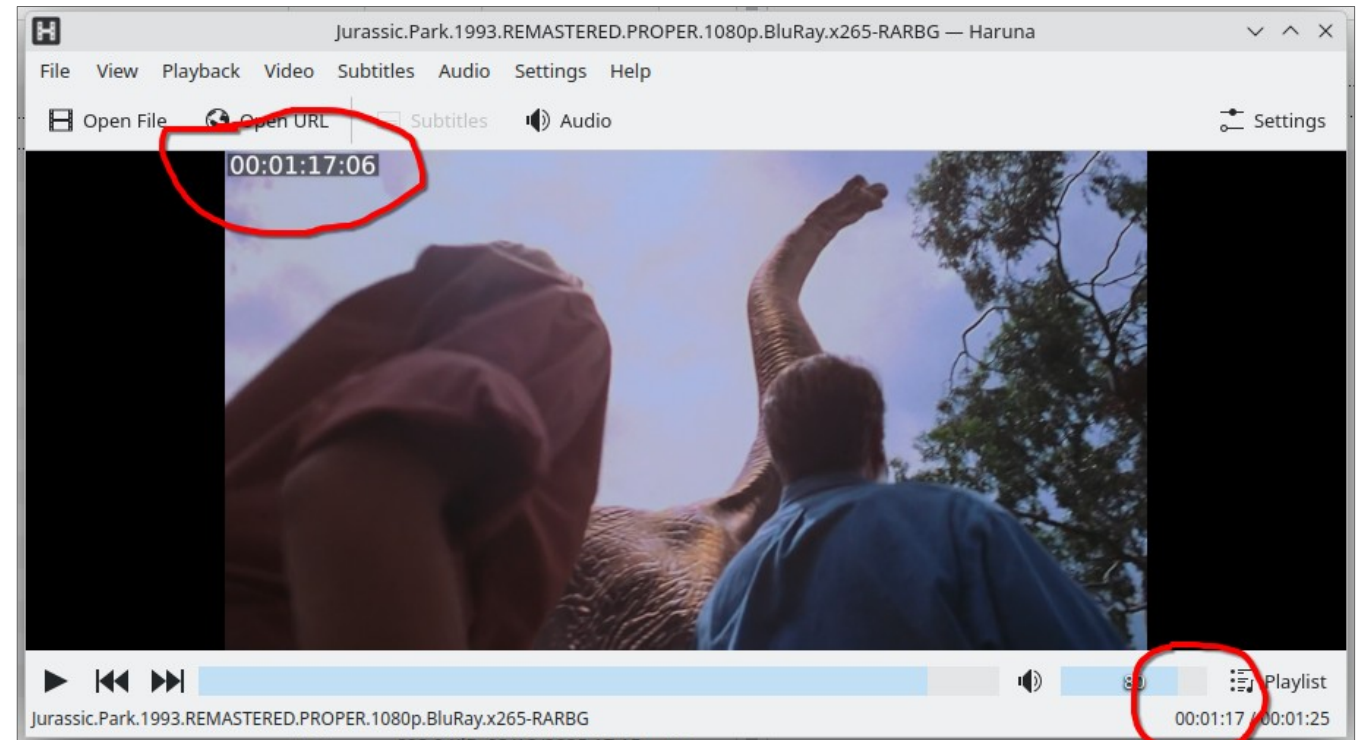
So we don't have any additional logic to wrap around it within the container.

No need to create a `app/processor.py` file.

To iterate quickly, it's usually best to first test the operator on a single, small file. Here we filter (-f) for the first clip of Jurassic Park.

```
FRAMESENSE_DOTENV_PATH='workshop/.env' python framesense.py timecode_clip_ffmpeg -f 00.19.48-85 -r
```

Open the output file with a video player, and check that the timecode matches the player's time at different moments.



```
operators > timecode_clip_ffmpeg > {} params.json > ...
```

```
1  {  
2  |    "fontsize": 24  
3  |  
4  |}
```

```
fontsize = self.get_param('fontsize')  
rate = self._get_video_frame_rate(clip_path)  
vf = (  
    f"drawtext=",  
    f"text='':"  
    f"timecode='00\\:00\\:00\\:00':"  
    f"rate={rate}:"  
    f"font='DejaVu Sans':"  
    f"x=10:y=10:"  
    f"fontsize={fontsize}:"  
    f"fontcolor=white:"  
    f"box=1:"  
    f"boxcolor=black@0.5:"  
    f"boxborderw=5"  
)  
command = [  
    'ffmpeg', '-y', '-i', clip_path,  
    '-vf', vf,  
    '-c:a', 'copy',  
    output_path,  
)  
self._run_in_operator_container(command, [clip_path.parent, '/data'], same_user=True)
```

```
operator.py > _timecode_clip()
```

```
operators > timecode_clip_ffmpeg > ⓘ README.md > ⓘ # timecode_clip_ffmpeg > ⓘ ## Input
1  # timecode_clip_ffmpeg
2
3  ## Input
4
5  * clips (C/C.mp4)
6
7  ## Output
8
9  * timecoded clips (C/C_timecoded.mp4)
10
11 ## Method
12
13 Uses ffmpeg's drawtext filter to overlay a running SMPTE timecode (HH:MM:SS:FF) to the
    top left corner of every frame of a clip. The frame rate of the output video matches the
    source video.
14
15 ## Parameters
16
17 * `fontsize`: font size of the timecode text (default: 24)
18
19 ## Run if
20
21 The timecoded clip (C/C_timecoded.mp4) does not already exist.
22
23 ## Redo (-r)
24
25 Will recreate the timecoded clip with a fresh ffmpeg pass.
26
```

Full working example

A fully working implementation of that operator is available in the `timecode_clip_ffmpeg` branch of the FrameSense repository.

```
git checkout timecode_clip_ffmpeg
```

How could it be improved?

1. Add a parameter for the **display location** (top left, top right, bottom, ...)
2. Add a parameter for up to 5x **faster processing** (e.g. ffmpeg -preset ultrafast)
3. Parameter request **absolute** time codes (w.r.t. overall video), instead of being relative to the start of each clip